

二重エージェントのジレンマ

- 時間制限: 12 minutes.
- ストレージ: 5 GB
- 環境: GPU 1基 (≈16 GB VRAM) 、インターネット接続なし
- 解答サイズ: `solution.ipynb` ≤ 1 MB
- ベースラインスコア: 0

Astanaの国立AIセンターでは、2つのコンピュータモデル、Model R (ResNet-18) とModel V (ViT-Tiny) が写真を分析しています。現在、両モデルは完璧に動作しており、正解率100%を達成し、すべての画像について同じ判定をしています。両者の賢い「頭脳」が実際にどれほど異なるかを試すため、主任科学者はあなたに課題を与えます。それぞれの写真のピクセルに、ほとんど見えないほど小さな変更を加え、Model RとModel Vの判定を完全に食い違わせてください。



1. 課題

2つの学習済み画像分類器が同じ画像を見ます。この課題で提供される画像に対して、両分類器は正解率 (accuracy)100%を達成します。

- **Model R:** `torchvision.models.resnet18` (CNN、ResNet18) 。
- **Model V:** `timmのvit_tiny_patch16_224` (Transformer、ViT-Tiny) 。

あなたの課題は、それぞれの画像に小さな変更（「摂動」）を作成し、2つのモデルの判定を食い違わせることです。各画像について、**2つの異なる摂動**を作成しなければなりません。

- **Type A:** 追加後もModel Rは画像を正しく分類しますが、Model Vは誤って分類します。
- **Type B:** 追加後もModel Vは画像を正しく分類しますが、Model Rは誤って分類します。

各摂動は、気づきにくいほど小さくしなければなりません。摂動が小さいほど高いスコアが得られます（Section 5を参照）。摂動は元の画像に対して、ピクセルレベルで直接適用されます。

2. 公開データ

この課題では、`train` (100 images) と `test_public` (100 images) という2つのsplitに分けられた画像セットが提供され、各splitにはさまざまな解像度の画像が含まれています。すべての画像はImageNet-1Kの1000 classesからのものであり、Model RとModel Vは両方のsplitで正解率100%を達成します。

以下のファイルが提供されます。

```
train/images/*.png          # 100 images in PNG format
train/labels.json           # maps each image index to its correct class
test_public/images/*.png    # 100 images in PNG format
test_public/labels.json     # maps each image index to its correct class
```

採点時には、あなたの`dataset/test_public/`フォルダは、公式採点用の2つの非公開画像セット（`test_leaderboard_a`および`test_leaderboard_b`）に透過的に置き換えられます。それぞれには、PNG形式の**100 images**とラベルファイルが含まれます。

注: この課題では、**test datasetsのラベルにアクセスできます。**

3. 出力形式

各画像について、2つのファイルを生成しなければなりません。

```
{index}_a.pt    # Type A perturbation
{index}_b.pt    # Type B perturbation
```

- `{index}` (0、1、2、...) は、`datasets`内の画像名と一致します。
- 各ファイルは、**`torch.save`で保存された単一のtensor**です。そのshapeは $3 \times H \times W$ でなければならない、 H と W は、その画像の**元の**解像度（ 224×224 ではありません）と一致しなければなりません。
- **コードは、`submission.zip`という1つのZIPファイルだけを生成する必要があります。**すべての`.pt`ファイルを、格納フォルダやサブディレクトリを設けず、ZIPアーカイブの最上位に配置してください。

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

出力形式に問題がある場合、notebookが警告します。

4. 制約

- **モデル** : `torchvision.models.resnet18(pretrained=True)` と `timm.create_model('vit_tiny_patch16_224', pretrained=True)` を使用しなければなりません。他の学習済みモデルは使用できません。
- **変換pipeline（評価時に強制）** : `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` で詳細を確認してください。
- **摂動の解像度**: 元のraw imageの解像度（224×224ではありません）と一致しなければなりません。tensorは、変換pipelineの前にraw imageへ追加されます。
- **出力形式**: .ptファイルのみです。PNG/JPGは不可です。tensorはraw imageに追加され、前処理の前にピクセル値が[0, 1]にクリップされます。
- **ファイル命名**: フラットに配置し、厳密に{index}_a.pt / {index}_b.pt形式に従ってください。zip内にサブディレクトリを含めてはいけません。
- **ライブラリ**: torch、torchvision、timm。

5. 採点

最終スコアは次のように計算されます。 M をsplit内の画像数、 $Score_A$ を成功したType A摂動の数、 $Score_B$ を成功したType B摂動の数とします。

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PFは、ノルムが大きい摂動にペナルティを科し、性能の上限付近で非常に敏感になるよう設計された関数です。これは0.5から1の範囲に制限されています。完全な実装は、`solution.ipynb`のSection 8で確認できます。

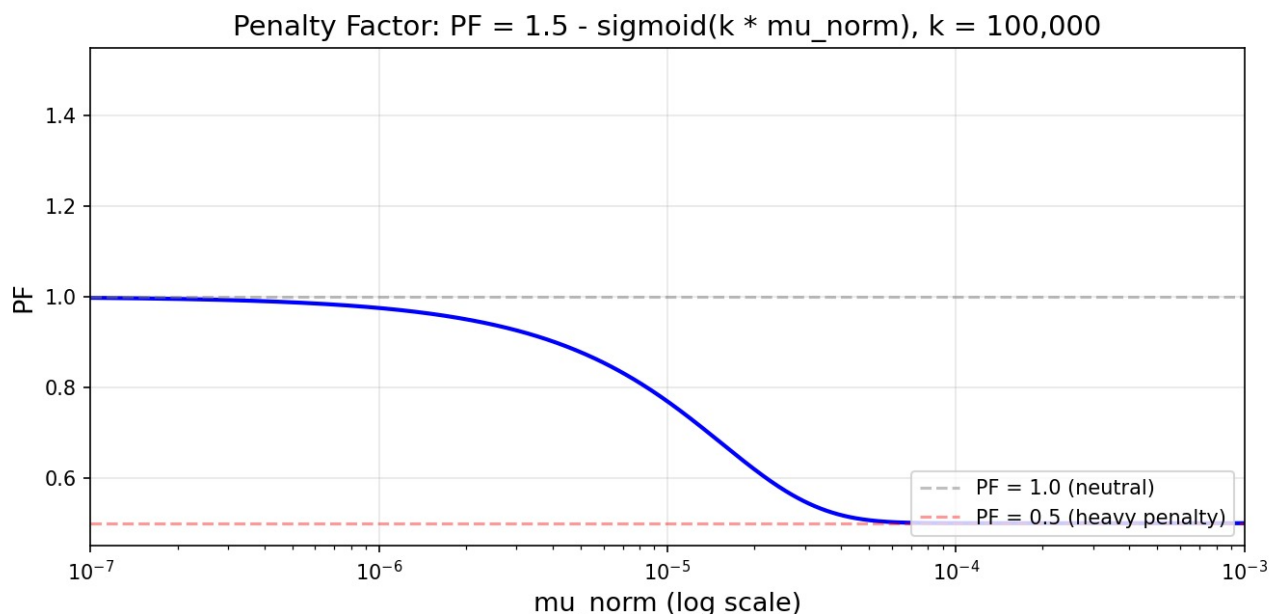


図: ペナルティ関数の曲線。

6. 提出物の確認

notebookには、形式上の問題がある場合に警告するチェックが、`solution.ipynb` notebookのSection 7にあります。

7. ローカルテスト

`solution.ipynb`には、完全に動作する例が含まれています。これは公開データ、両方のモデル、および公式scorerを読み込み、提出用ZIPファイルを書き出します。開始する前に読んでください。

8. 提出方法

- 変更内容を`solution.ipynb`に保存してください。
- JupyterLabの左側のsidebarでGit tabを開いてください。
- `solution.ipynb`を**Stage**してください（その横にある+ icon）。
- commit messageを入力し、**Commit**をクリックしてください。
- 上向き矢印付きのcloud iconをクリックしてpushしてください。
- このContest pageに戻り、**Submit**をクリックしてください。

`solution.ipynb`という名前のファイルだけを提出してください。